

Coffee Kiosk Architecture Justification

Lab 3 | Component Modelling and Architectural Pattern Selection

Architecture Selection

We chose **Layered Architecture for the Self-Service Coffee Kiosk System**. The kiosk is a compact application with one user interface, a small ordering workflow, credit-card payment, local menu data, and receipt-printer integration. Separating these responsibilities into presentation, business, and data/device layers provides enough structure without the deployment and networking overhead of distributed services.

Architectural Style Analysis

Style	Fit for this scenario	Main tradeoff
Layered	Strong fit: simple module boundaries and direct local calls suit one kiosk.	Some cross-layer call overhead.
Microservices	Supports independent scaling, but the kiosk has too few functions to justify distribution.	Network, deployment, and consistency complexity.
Client-Server	Central control could help a fleet, but the stated scenario is a self-contained kiosk.	Server availability becomes a bottleneck and failure point.

Two Scenario Specific Reasons

1 Clear responsibility boundaries. The Touchscreen UI can change independently of the Order Manager, while menu storage, payment authorization, and printer control remain behind stable interfaces. This makes the small system easier to test, replace, and maintain.

2 Practical hardware and data integration. The business layer coordinates an order without knowing SQL details or printer commands. The Menu Repository and Receipt Printer Adapter isolate those technologies, reducing the impact of a database or printer-model change.

Security Advantage

Payment isolation. Only the Order Manager can request payment through the Payment API. The Touchscreen UI and Menu Repository never receive raw card data; the Payment Service validates the request and returns only an authorization result and transaction reference. This narrows the sensitive-data boundary and supports input validation and least-privilege access.

Performance Benefit

Low-latency local flow. Presentation, ordering, menu lookup, and printer coordination can use in-process calls or local adapters. Avoiding multiple network hops gives responsive touch interactions and predictable checkout time. Frequently read menu and pricing data can also be cached in the business layer while the repository remains the source of truth.

Component and Interface Summary

Five components are used: Touchscreen UI, Order Manager, Payment Service, Menu Repository, and Receipt Printer Adapter. Their four assembly interfaces are Order API, Payment API, Menu Query Interface, and Printer Port.